

UNIVERSIDADE FEDERAL DE ALAGOAS

INSTITUTO DE COMPUTAÇÃO

DISCIPLINA: PROGRAMAÇÃO 2

ALUNO: JOÃO VICTOR DUARTE DO NASCIMENTO

RELATÓRIO DE ANÁLISE DO PROJETO JACKUT: MILESTONE 2

Repositório: github.com/johnUFAL/Jackut

MACEIÓ, 2025

1. Introdução

Este documento é a segunda versão do relatório de análise do projeto Jackut. Incorpora as mudanças introduzidas nos dois commits mais recentes (75d1f20 e cb71a92), que adicionaram suporte à User Story 9 (remoção de conta), documentação Javadoc completa e correções de bugs previamente apontados na versão 1 deste relatório.

A estrutura do relatório é a mesma da versão anterior: Avaliação (virtudes e fraquezas), Design Patterns, Diagrama de Classes e Conclusão com avaliação objetiva. As seções que sofreram alteração em relação à v1 estão marcadas com [ATUALIZADO].

Projeto	Jackut – Rede Social
Versão do relatório	2 (pós-push de US9 e Javadoc)
Últimos Commits	75d1f20 (feat: us9) e cb71a92 (docs+feat: us9 + javadoc)
Últimas classes	MensagemJackut
Últimos métodos	removerUsuario (ServicoDePerfil + Facade)
Bugs corrigidos	ServicoDeRecados (ordem), ServicoDeAmizades (ordem), typos adicionar*
Testes cobertos	us1 a us9 (8 scripts de aceitação)

2. Mudanças Identificadas nos Novos Commits [ATUALIZADO]

A tabela a seguir consolida todas as alterações encontradas ao comparar o estado anterior do repositório (commit 99267a0) com o estado atual (commit cb71a92).

Elemento	Tipo de mudança	Descrição
MensagemJackut	Novo – Value Object	Nova classe que encapsula remetente + texto de recados e mensagens, substituindo String puro nas filas.
Destinatario (interface)	Alterado	Assinatura receberRecado alterada: agora recebe (remetente, recado) além do texto, habilitando rastreamento do autor.

Usuario.recados / mensagens	Alterado	Tipo das filas mudou de List<String> para List<MensagemJackut>, integrando o novo Value Object.
Usuario.adionarFa/Inimigo	Corrigido – Typo	Métodos renomeados para adicionarFa e adicionarInimigo. Padrão de nomenclatura restaurado.
ServicoDePerfil.removerUsuario	Novo – Cascata	Método que remove um usuário e todos os seus rastros: recados enviados, vínculos de amizade, comunidades que possuía, etc.
Facade.removerUsuario	Novo	Delegação adicionada na Facade para expor removerUsuario ao EasyAccept (US9).
Facade.adicionarComunidade	Corrigido – Typo	Método anteriormente chamado adicionarComunidade corrigido para adicionarComunidade.
ServicoDeRecados.enviarRecado	Corrigido – Bug	Verificação de inimigo agora ocorre ANTES de entregar o recado. Bug de ordem da v1 corrigido.
ServicoDeAmizades.adicionarAmigo	Corrigido – Bug	Verificação de inimigo movida para antes da verificação de convite. Ordem lógica restaurada.
ServicoDeRelacionamentos.adicionarPaquera	Alterado	Recado de match agora usa receberRecado("Jackut", texto) em vez de receberRecado(texto), passando remetente.
Javadoc	Novo – Documentação	Todos os arquivos .java (Facade, Servicos, Repositório, Modelos, Exceções) receberam Javadoc completo.
tests/us9_1.txt e us9_2.txt	Novo – Testes	Scripts de aceitação para a US9 (removerUsuario) adicionados ao repositório.

3. Avaliação

3.1 Virtudes do Projeto [ATUALIZADO]

Value Object MensagemJackut (novo)

A introdução da classe MensagemJackut é a melhoria de design mais significativa nesta versão. Anteriormente, recados e mensagens eram armazenados como List<String> — uma String simples sem origem identificada. Com MensagemJackut, cada mensagem carrega consigo o login do remetente (remetente) e o conteúdo (texto), tornando possível

filtrar por autor durante a remoção de conta (`removerUsuario` usa `removelf(m -> m.getRemetente().equals(id))`). Esse é o padrão Value Object do DDD: um objeto imutável que encapsula dados semanticamente relacionados.

A justificativa técnica é clara: sem o remetente encapsulado, seria impossível implementar a remoção em cascata de recados de um usuário excluído sem recorrer a heurísticas frágeis. A refatoração mostra que o design evoluiu corretamente para suportar o novo requisito.

Remoção em Cascata (novo)

O método `removerUsuario` no `ServicoDePerfil` implementa um delete em cascata completo: remove recados e mensagens enviados pelo usuário das caixas de terceiros; remove o usuário de todas as listas de relacionamento (amigos, convites, ídolos, fãs, paqueras, inimigos); exclui as comunidades das quais o usuário era dono, retirando todos os outros membros; e, por fim, remove o próprio registro do repositório.

A concentração dessa lógica em `ServicoDePerfil` é a decisão certa: é o serviço que conhece o ciclo de vida completo de uma conta de usuário. Colocar essa lógica na `Facade` ou espalhar entre os outros serviços quebraria a coesão existente.

Bugs Corrigidos (vs. versão 1)

Os dois bugs de ordem relatados na versão anterior foram corrigidos. No `ServicoDeRecados`, a verificação de inimigo agora precede a entrega do recado — nenhum conteúdo chega ao destinatário se houver bloqueio. No `ServicoDeAmizades`, a verificação de inimigo também foi movida para o início do método `adicionarAmigo`, antes de qualquer verificação de convite, o que é semanticamente correto: um inimigo não pode nem tentar uma amizade.

Tipos Corrigidos e Nomenclatura Uniforme

Todos os tipos identificados na versão anterior foram corrigidos: `adionarFa` tornou-se `adicionarFa`, `adionarInimigo` tornou-se `adicionarInimigo`, e `adionarComunidade` (na `Facade`) tornou-se `adicionarComunidade`. A nomenclatura do projeto agora é consistente, facilitando leitura, autocompletar na IDE e manutenção futura.

Documentação Javadoc Completa

Esta versão adicionou Javadoc em todos os arquivos Java do projeto: `Facade`, todos os `Servicos`, `RepositorioJackut`, todos os modelos (`Usuario`, `Comunidade`, `EntidadeJackut`, `MensagemJackut`, `Destinatario`) e todas as classes de exceção. A documentação explica o propósito de cada classe, o contrato de cada método, os

parâmetros e as exceções lançadas, elevando significativamente a qualidade documental do projeto.

3.2 Fraquezas Remanescentes

Classe Usuario ainda Acumula Responsabilidades

Apesar das melhorias, a classe Usuario continua como um objeto-deus. Agora contém 12 coleções, métodos de negócio (possuiConviteDe, aceitarConvite, registrarConviteEnviado) e a lógica de formatação de listas. A adição de MensagemJackut e da remoção por remetente reforçou ainda mais essa classe como centro do sistema. Uma refatoração ideal separaria a gestão dos relacionamentos em objetos de domínio próprios.

Duplicação em formatarListaDeMembros (Comunidade)

A classe Comunidade ainda possui seu próprio método formatarListaDeMembros com o mesmo algoritmo de formatarLista presente em Usuario. Com a introdução de MensagemJackut, ficou ainda mais evidente que uma classe utilitária compartilhada (ex: FormatadorJackut.formatarLista(List<String>)) eliminaria essa duplicação de forma elegante, respeitando o princípio DRY.

Acoplamento Direto aos Mapas Internos em removerUsuario

O método removerUsuario acessa diretamente `repo.getUsuarios().values()` e `repo.getComunidades().values()` para iterar e modificar as coleções internas do repositório. Isso quebra o encapsulamento do RepositorioJackut: o serviço manipula a estrutura interna dos mapas sem passar pelos métodos de acesso do repositório. O ideal seria adicionar métodos como `repo.removerMembrosDeTodos(id)` ou `repo.excluirComunidadesDoUsuario(id)` ao próprio repositório, mantendo a coesão da camada de dados.

Sessão Ainda Retorna o Próprio Login

A questão levantada na versão anterior permanece: `abrirSessao` retorna o login do usuário como ID de sessão. Isso é adequado para os testes de aceitação do EasyAccept, mas semanticamente incorreto para um sistema real. Não houve progresso neste ponto, o que é compreensível dado que o foco dos commits foi a US9.

4. Design Patterns Identificados [ATUALIZADO]

Os padrões identificados na versão 1 (Facade, Repository, Service Layer, Dependency Injection, Template Method/Polimorfismo e Serialização XML) permanecem válidos. Esta seção adiciona os novos padrões introduzidos nesta versão.

4.1 Value Object (DDD) – MensagemJackut (novo)

Localização: `br.ufal.ic.p2.jackut.models.MensagemJackut`

Um Value Object é um objeto que descreve uma característica do domínio e não possui identidade própria — dois objetos com os mesmos valores são equivalentes. `MensagemJackut` encapsula dois campos relacionados (remetente e texto) que antes viajavam como String separadas ou eram perdidos na conversão. A classe implementa `Serializable` (necessário para o `XMLEncoder`) e segue o padrão `JavaBean` com construtor vazio + getters/setters.

Justificativa: sem o remetente encapsulado junto ao texto, o requisito de remoção em cascata da US9 seria inviável de forma limpa. A decisão de criar `MensagemJackut` foi tecnicamente motivada por esse requisito, mostrando bom uso do princípio de design orientado ao domínio: o modelo reflete as necessidades do negócio.

4.2 Cascade Delete (Arquitetural) – `removerUsuario` (novo)

Localização: `ServicoDePerfil.removerUsuario`

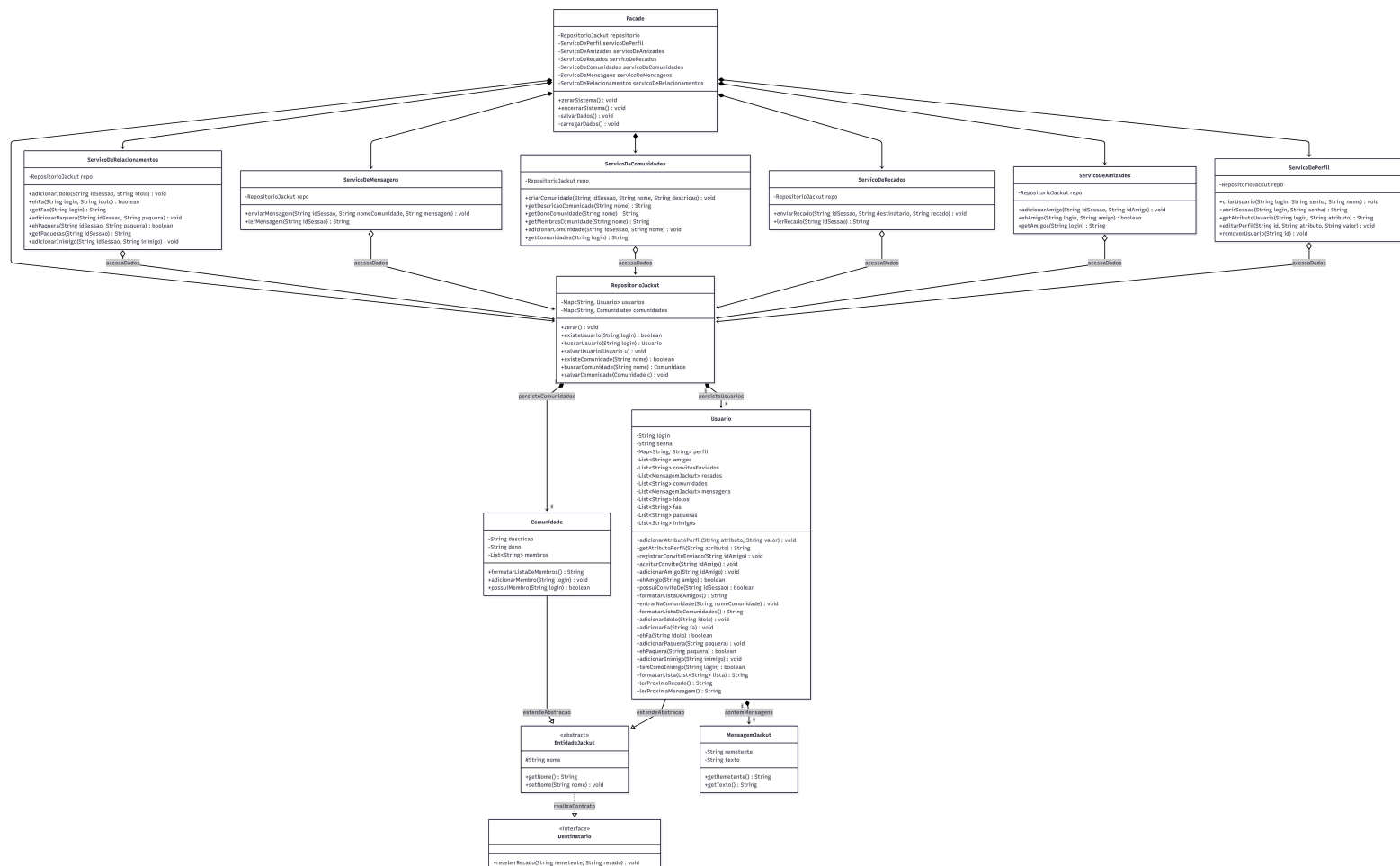
O padrão Cascade Delete garante que a remoção de uma entidade raiz propague a exclusão para todas as entidades dependentes. Aqui, remover um usuário dispara: limpeza das filas de outros usuários, remoção de listas de relacionamento, exclusão das comunidades do usuário e atualização dos perfis dos membros dessas comunidades.

Justificativa: em bancos de dados relacionais, esse comportamento seria declarado via `ON DELETE CASCADE` nas chaves estrangeiras. Como o sistema usa objetos em memória sem um ORM, a lógica de cascata precisa ser implementada manualmente no código. A concentração dessa lógica no `ServicoDePerfil` é a escolha correta pelo princípio GRASP Expert: o serviço que conhece o ciclo de vida do usuário é o mais apto a gerenciar sua remoção.

5. Diagrama de Classes [ATUALIZADO]

O diagrama atualizado (gerado como SVG interativo no relatório visual) reflete as seguintes mudanças estruturais em relação à versão anterior:

- Nova classe MensagemJackut (Value Object), com remetente e texto, usada por Usuario nas listas recados e mensagens.
- Interface Destinatario atualizada: receberRecado agora recebe dois parâmetros (remetente, recado).
- ServicoDePerfil com método removerUsuario em destaque, indicando remoção em cascata.
- Métodos corrigidos destacados com marcação de mudança: adicionarFa, adicionarInimigo, adicionarComunidade.
- Todos os serviços e o repositório com ícone de Javadoc completo.



6. Refatoramentos Remanescentes

Com as correções desta versão, a lista de refatoramentos da v1 foi reduzida. Os itens abaixo permanecem pendentes:

6.1 Encapsular Acesso ao Repositório em `removerUsuario`

O método `removerUsuario` acessa `repo.getUsuarios().values()` e `repo.getComunidades().values()` diretamente para iterar e modificar as coleções internas. Isso viola o encapsulamento do `RepositorioJackut`. A solução é adicionar métodos de suporte no próprio repositório (ex: `removerMembrosDeTodos`, `listarComunidadesPorDono`) que realizam essas operações internamente, mantendo o repositório como a única porta de acesso aos dados.

6.2 Utilitário Compartilhado de Formatação de Listas

O algoritmo `{item1,item2,...}` ainda está duplicado em `Usuario` (`formatarLista`) e em `Comunidade` (`formatarListaDeMembros`). Um método estático em uma classe utilitária compartilhada (ex: `JackutUtils.formatarLista(List<String>)`) eliminaria essa duplicação. A `Comunidade` poderia usar o utilitário em vez de reimplementar o mesmo loop.

7. Conclusão

Esta versão do relatório reflete um progresso concreto em relação à v1. Os dois bugs de ordem reportados foram corrigidos, os typos de nomenclatura foram eliminados, a documentação Javadoc foi adicionada a todo o projeto e a User Story 9 foi implementada com boas decisões de design: a criação de `MensagemJackut` como Value Object e a implementação de remoção em cascata concentrada no `ServicoDePerfil`.

O projeto demonstra capacidade de evoluir com qualidade: cada novo requisito resultou em melhorias estruturais no código, não apenas na adição de código novo sem refatoração. As fraquezas remanescentes são gerenciáveis e não comprometem a qualidade geral do sistema.

7.1 Avaliação Objetiva Atualizada

Critério	Nota	Justificativa
Qualidade da documentação (Javadoc)	9,5	Javadoc completo em todos os arquivos. Apenas um typo residual (@param) em Comunidade.
Qualidade do design (padrões e princípios)	8,5	MensagemJackut como Value Object e remoção em cascata são boas adições. Usuario ainda concentra demais.
Qualidade do código (legibilidade, bugs)	9,0	Bugs de ordem corrigidos, typos eliminados. Acoplamento ao mapa interno em removerUsuario é o ponto pendente.
Organização de pacotes	9,0	Inalterada: continua bem estruturada em models, controllers e exceptions.
Testabilidade (EasyAccept)	9,5	US9 coberta com us9_1.txt e us9_2.txt. 8 de 8 scripts de aceitação presentes.